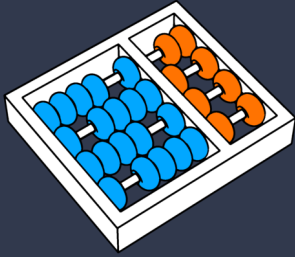




UNICAMP



Model Context Protocol (MCP)

Prof. Dr. Julio Cesar dos Reis
dosreis@unicamp.br

Agenda

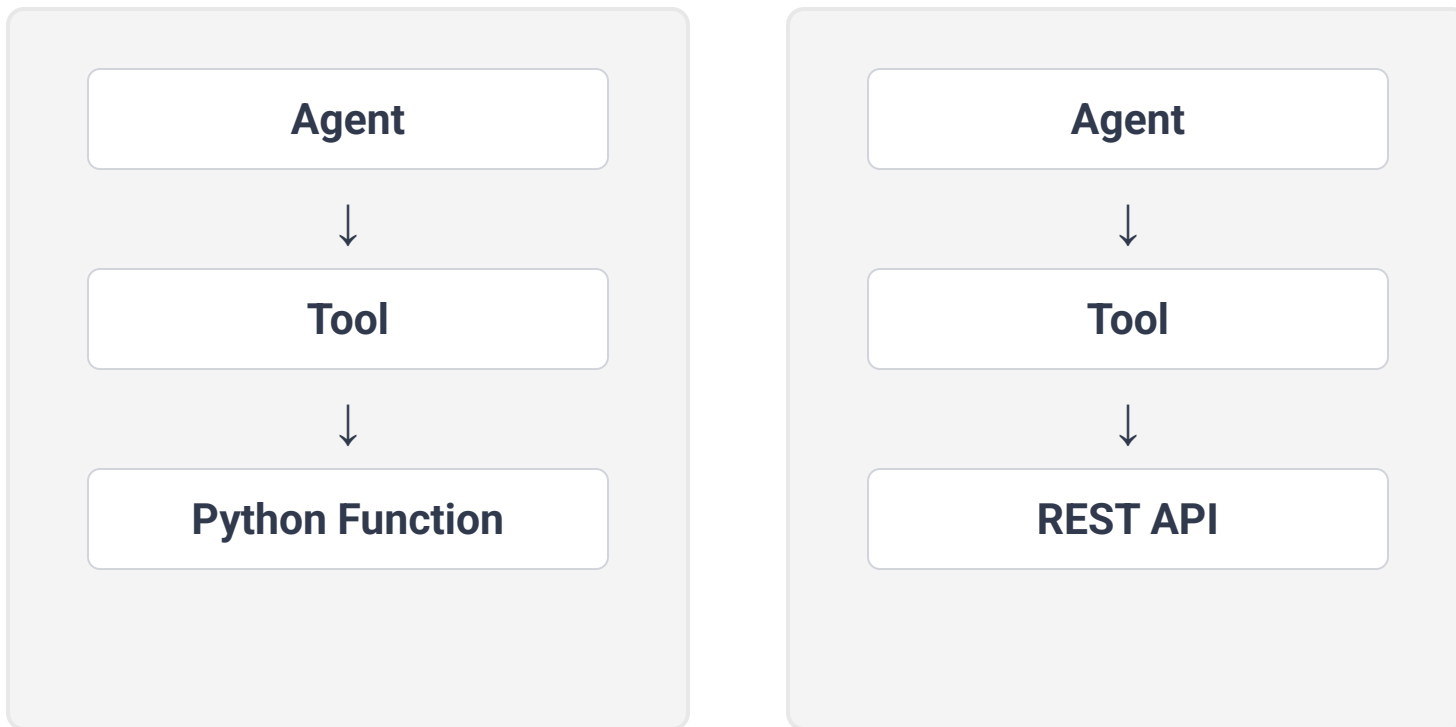
- ❑ Definição do conceito
- ❑ Componentes principais
- ❑ Arquitetura
- ❑ Segurança e controle de acesso

Motivação

- ❑ Até agora todas as nossas Tools estavam locais
- ❑ Mas será que isso é necessário?

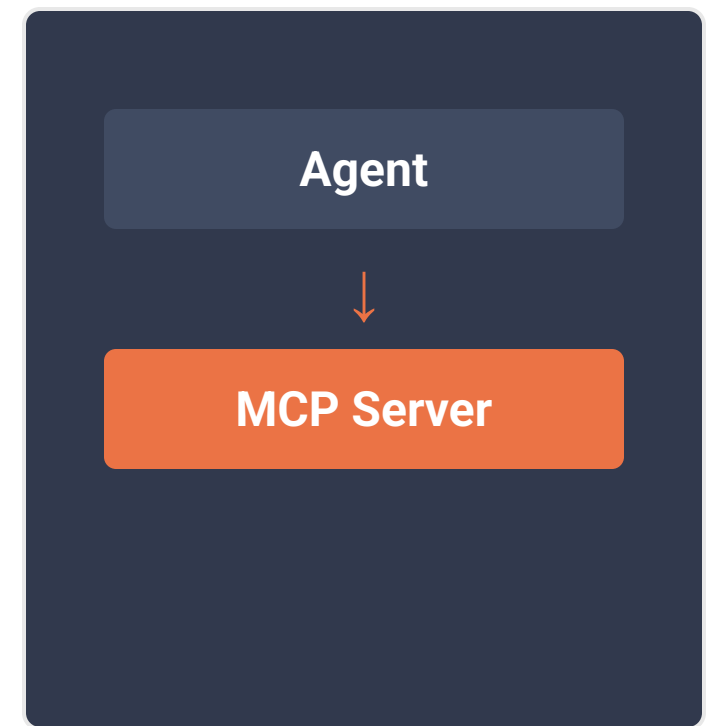
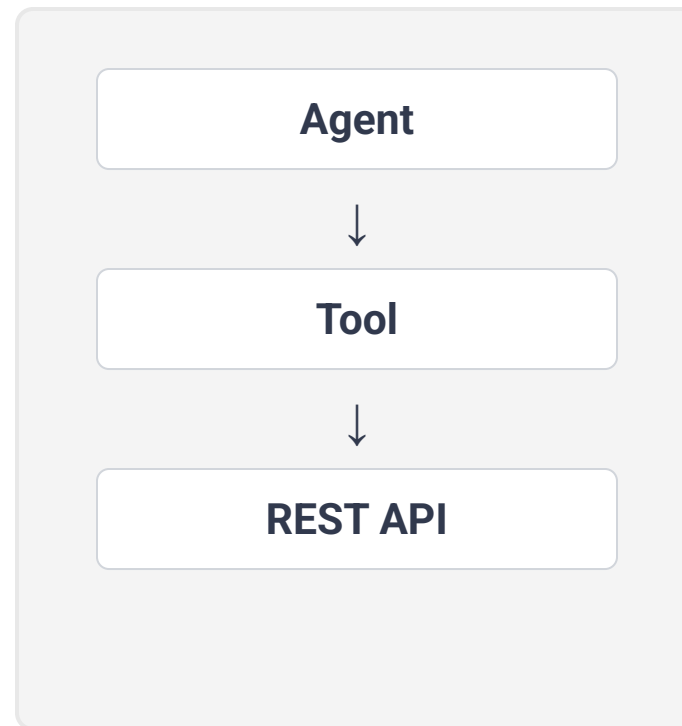
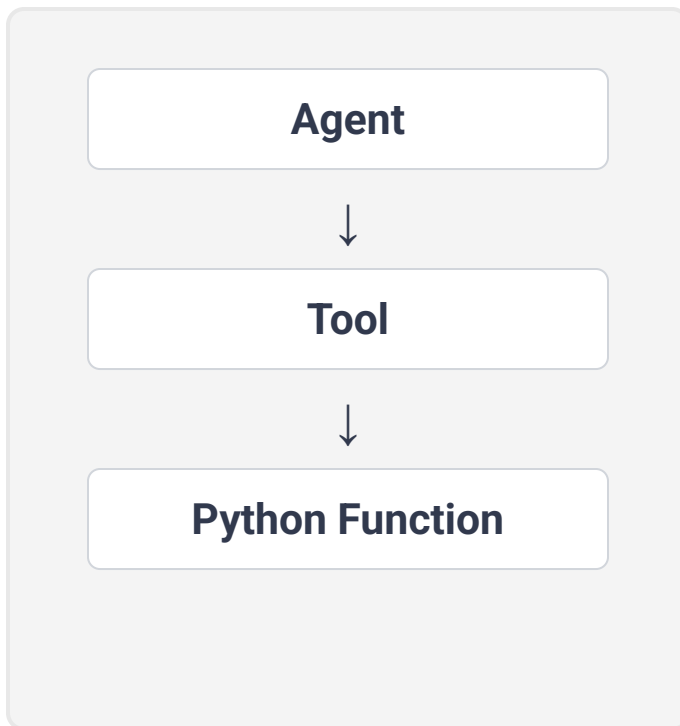
Motivação

- ❑ A interface da Tool permanece exatamente igual
- ❑ O que muda é apenas sua localização



Motivação

- ❏ A interface da Tool permanece exatamente igual
- ❏ O que muda é apenas sua localização



Model Context Protocol (MCP)

- ❏ Protocolo aberto que define como modelos de linguagem e agentes podem acessar recursos externos de forma
 - padronizada
 - segura
 - interoperável
- ❏ Uma infraestrutura para construir ecossistemas de agentes interoperáveis

MCP como USB-C para Agentes

Antes: Complexidade e Fragmentação

Banco de Dados → Interface A

CRM → Interface B

ERP → Interface C

Filesystem → Interface D

Sem um padrão comum, cada agente precisa aprender e implementar dezenas de interfaces diferentes.

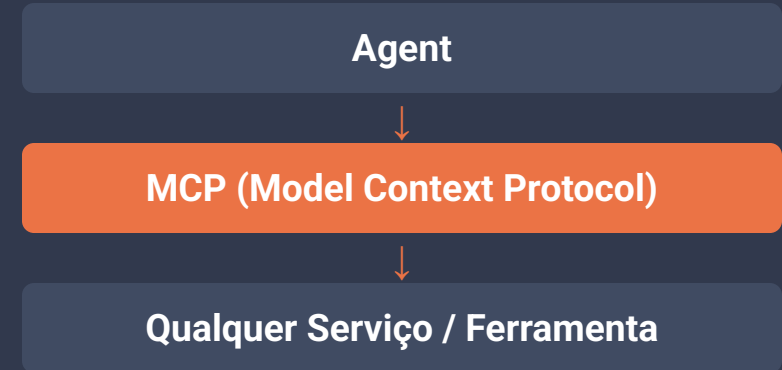
MCP como USB-C para Agentes

Antes: Complexidade e Fragmentação

Banco de Dados → Interface A
CRM → Interface B
ERP → Interface C
Filesystem → Interface D

Sem um padrão comum, cada agente precisa aprender e implementar dezenas de interfaces diferentes.

Depois: Padronização Unificada



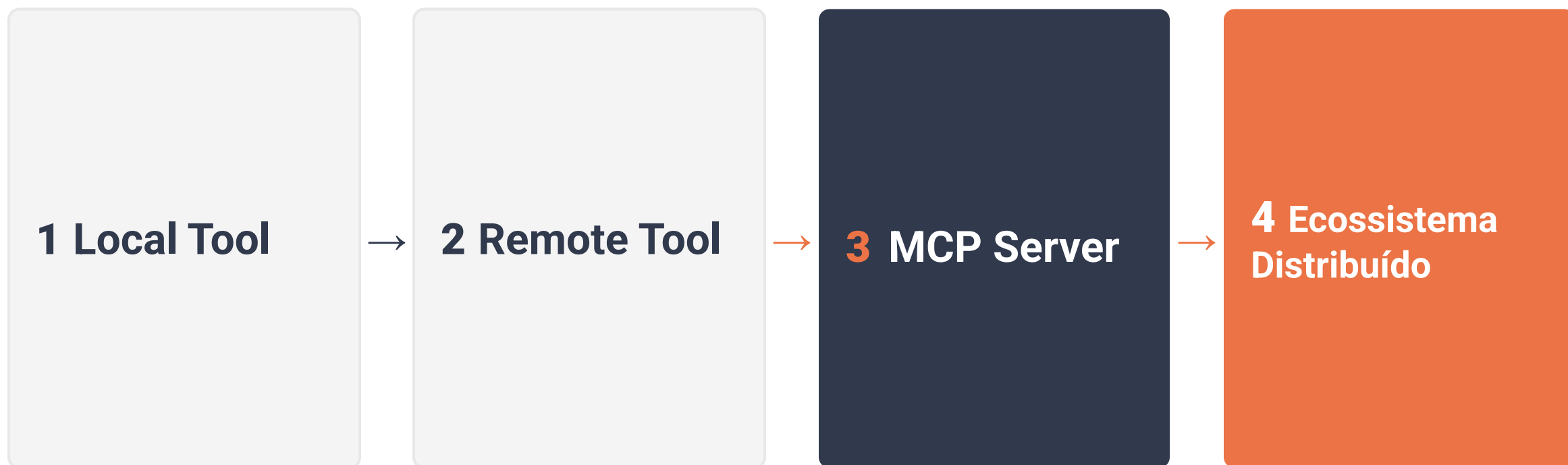
O MCP funciona exatamente como um **USB-C para IA**, criando uma interface única de conexão universal e simplificada.

Padronização

- ❏ MCP padroniza para qualquer agente
 - ferramentas
 - recursos
 - prompts
 - capacidades

Evolução

- ❑ MCP não substitui Tools
- ❑ MCP distribui Tools



Model Context Protocol (MCP)

Padronização

Define contratos comuns para que agentes acessem dados e ferramentas sem acoplamento direto

Interoperabilidade

Permite que diferentes agentes, modelos e ferramentas se comuniquem usando o mesmo protocolo

Extensibilidade

Facilita a adição de novos recursos e serviços ao ecossistema de agentes sem reescrever integrações

Sem e Com MCP

Sem MCP

- ❑ Cada agente cria sua própria integração
- ❑ Interfaces inconsistentes entre ferramentas
- ❑ Retrabalho ao adicionar novos serviços
- ❑ Difícil de auditar e monitorar

Sem e Com MCP

Sem MCP

- ❑ Cada agente cria sua própria integração
- ❑ Interfaces inconsistentes entre ferramentas
- ❑ Retrabalho ao adicionar novos serviços
- ❑ Difícil de auditar e monitorar

Com MCP

- ❑ Protocolo único para todos os recursos
- ❑ Contratos explícitos e padronizados
- ❑ Novos serviços plugáveis sem reescrita
- ❑ Observabilidade centralizada

Componentes Principais

Três componentes centrais

- ❑ MCP organiza as capacidades expostas por um servidor em três tipos distintos de primitivas.

Três componentes centrais

Prompts

Templates de interação reutilizáveis que o servidor disponibiliza para padronizar como o agente formula pedidos ou usa recursos específicos.

Exemplo: template para resumo de documento, prompt para geração de relatório

Três componentes centrais

Prompts

Templates de interação reutilizáveis que o servidor disponibiliza para padronizar como o agente formula pedidos ou usa recursos específicos.

Exemplo: template para resumo de documento, prompt para geração de relatório

Resources

Fontes de dados que o agente pode ler: arquivos, banco de dados, APIs de consulta. identificados por URIs e retornam conteúdo estruturado ou textual.

Exemplo: conteúdo de um arquivo, resultado de uma query SQL, resposta de uma API REST

Três componentes centrais

Prompts

Templates de interação reutilizáveis que o servidor disponibiliza para padronizar como o agente formula pedidos ou usa recursos específicos.

Exemplo: template para resumo de documento, prompt para geração de relatório

Resources

Fontes de dados que o agente pode ler: arquivos, banco de dados, APIs de consulta. Identificados por URIs e retornam conteúdo estruturado ou textual.

Exemplo: conteúdo de um arquivo, resultado de uma query SQL, resposta de uma API REST

Tools

“Funções” que o agente pode invocar para executar ações ou obter resultados computados. Possuem esquema de entrada e saída bem definido.

Exemplo: executar código, chamar API externa, criar registro em banco de dados

Arquitetura do MCP

Arquitetura do MCP

- ❏ MCP separa responsabilidades em três camadas
 - Host (**aplicação**)
 - Client (**intermediário**)
 - Server (**provedor de recursos**)

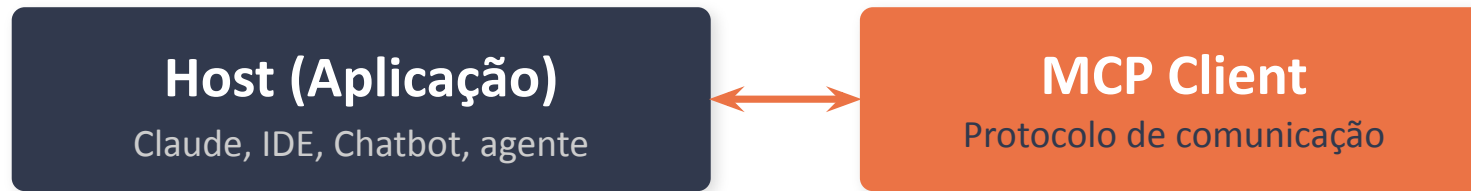
Host (aplicação)

Host (Aplicação)

Claude, IDE, Chatbot, agente

- ❏ **Host:** a aplicação que incorpora a LLM e inicia a sessão MCP

Client (intermediário)



- ❑ **Host:** a aplicação que incorpora a LLM e inicia a sessão MCP
- ❑ **MCP Client:** gerencia a conexão e traduz pedidos do host para o protocolo MCP

Client (intermediário)

- ❏ Por que existe o MCP Client?
- ❏ Responsabilidades:
 - Gerenciar conexões
 - Descobrir capacidades
 - Traduzir chamadas
 - Validar mensagens
 - Controlar autenticação
- ❏ Ele funciona como um *driver* entre o host e o servidor.

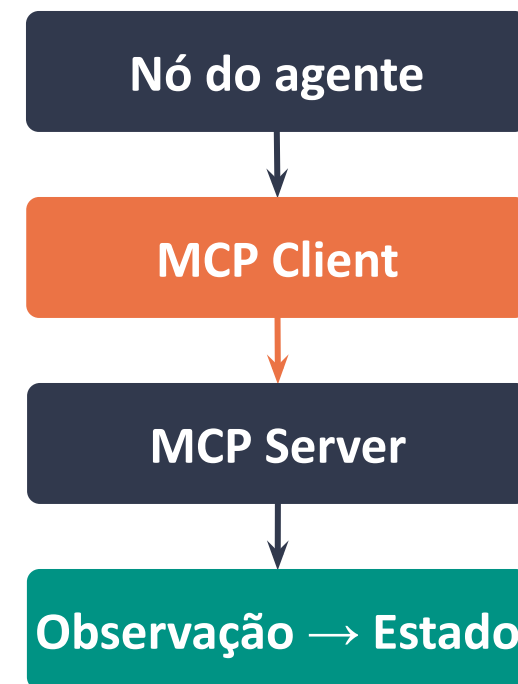
MCP Server



- ❑ **Host**: a aplicação que incorpora a LLM e inicia a sessão MCP
- ❑ **MCP Client**: gerencia a conexão e traduz pedidos do host para o protocolo MCP
- ❑ **MCP Server**: expõe resources, tools e prompts — pode ser local ou remoto

MCP como camada de integração do agente

- ❑ Em arquiteturas baseadas em LangGraph, o MCP atua como camada de abstração que padroniza o acesso do agente a recursos externos
- ❑ O nó de ação do grafo chama o MCP Client
- ❑ O Client “descobre” quais tools e resources estão disponíveis
- ❑ A tool é invocada com argumentos validados pelo esquema
- ❑ O retorno entra no estado do grafo como observação
- ❑ O agente decide o próximo passo com base na observação



Descoberta Dinâmica de Capacidades

Ao conectar-se ao servidor MCP

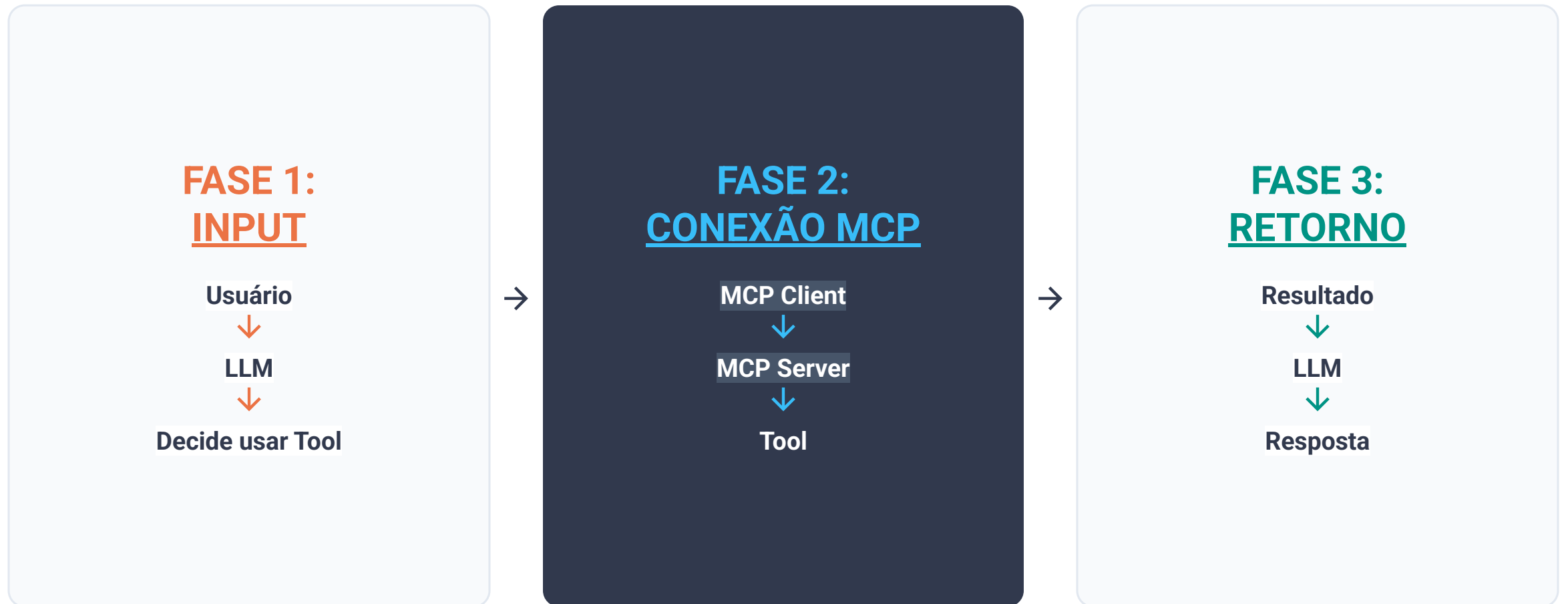
- a. O cliente lista **resources**
- b. O cliente lista **tools**
- c. O cliente lista **prompts**
- d. O agente decide o que usar

⚡ Não há *hardcoding*

🔄 O agente descobre capacidades em tempo de execução



Ciclo de execução MCP



Exemplo

- ❏ **Objetivo:** permitir que um LLM resolva serviços acadêmicos

Resources

- ❏ alunos://notas
- ❏ alunos://frequencia
- ❏ disciplinas://ementas

Tools

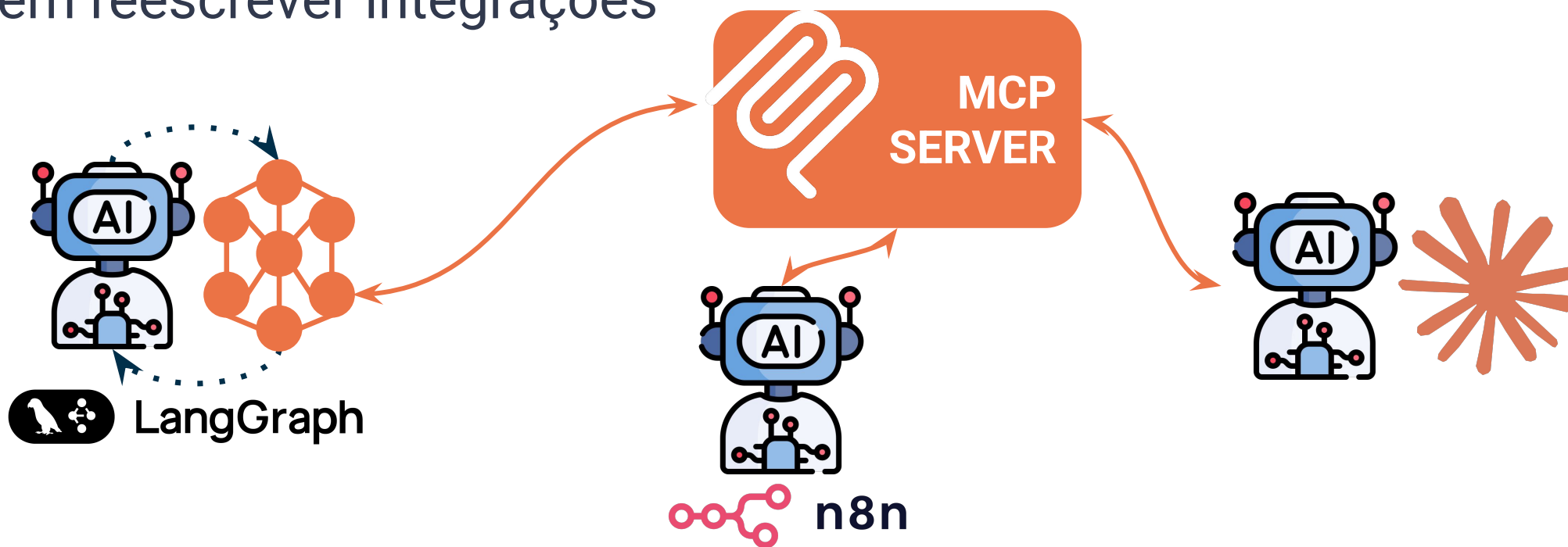
```
calcular_media()  
gerar_relatorio()  
enviar_email()
```

Prompt

“Gerar parecer acadêmico”

Interoperabilidade entre agentes

- ❑ Diferentes frameworks e modelos compartilham o mesmo conjunto de servidores de recursos
- ❑ Sem reescrever integrações



Padrões de uso do MCP com LangGraph

- ❏ A escolha do padrão depende do tipo de recurso, do nível de autonomia desejado e da necessidade de controle sobre o fluxo.

Padrão	Idéia	Quando usar
<u>Tool</u> via MCP	Nó do grafo chama tool via MCP Client	Integração a serviços externos
<u>Resource Reader</u>	Agente lê arquivos ou dados via resources MCP	Acesso a bases de conhecimento
<u>Prompt Templates</u>	Agente usa prompts do servidor para estruturar pedidos	Padronização de interações com modelos
MCP + Supervisor	Supervisor delega a sub-agentes via ferramentas MCP	Orquestração de tarefas compostas

Segurança e Controle de Acesso

Segurança e Controle de Acesso

- ❑ O MCP foi projetado para que o servidor controle quais recursos e ações são acessíveis, mantendo o princípio do menor privilégio

Segurança e Controle de Acesso

Controle pelo servidor

O server decide quais **tools** e **resources** expõe.
O agente não acessa diretamente nenhum sistema,
tudo passa pelo contrato MCP.

Segurança e Controle de Acesso

Controle pelo servidor

O server decide quais **tools** e **resources** expõe.
O agente não acessa diretamente nenhum sistema,
tudo passa pelo contrato MCP.

Validação de esquema

Os argumentos de tools são validados contra JSON Schema antes de qualquer execução, reduzindo erros e injeções.

Descrição de uma Tool

Contrato da Tool



Além do Nome

O LLM não vê apenas o nome da ferramenta. Ele lê a descrição detalhada para compreender o propósito e a utilidade da Tool.



Contrato Formal

O modelo recebe uma especificação rígida de entrada e saída, garantindo que as chamadas sejam validadas de forma estrita.

```
{  
  "name": "consultar_clima",  
  "description": "Obtém previsão meteorológica",  
  "inputSchema": {  
    "type": "object",  
    "properties": {  
      "cidade": {  
        "type": "string"  
      }  
    }  
  }  
}
```

Segurança e Controle de Acesso

Controle pelo servidor

O server decide quais **tools** e **resources** expõe. O agente não acessa diretamente nenhum sistema, tudo passa pelo contrato MCP.

Validação de esquema

Os argumentos de tools são validados contra JSON Schema antes de qualquer execução, reduzindo erros e injeções.

Autenticação e autorização

O transporte pode incorporar mecanismos de autenticação (OAuth, tokens).
Cada chamada é rastreável e auditável.

Segurança e Controle de Acesso

Controle pelo servidor

O server decide quais **tools** e **resources** expõe. O agente não acessa diretamente nenhum sistema, tudo passa pelo contrato MCP.

Autenticação e autorização

O transporte pode incorporar mecanismos de autenticação (OAuth, tokens).
Cada chamada é rastreável e auditável.

Validação de esquema

Os argumentos de tools são validados contra JSON Schema antes de qualquer execução, reduzindo erros e injeções.

Human-in-the-loop

O MCP permite que servidores marquem ações **sensíveis** como **exigindo confirmação** do usuário antes de executar.

Riscos sem MCP

- ❑ Execução arbitrária de código
- ❑ APIs sem validação
- ❑ Vazamento de credenciais
- ❑ Prompt Injection
- ❑ Tool Abuse

Impactos

- ❏ Reuso
- ❏ Interoperabilidade
- ❏ Escalabilidade
- ❏ Governança
- ❏ Economia de agentes

Síntese

- ❑ MCP é um protocolo aberto que padroniza como agentes acessam recursos externos
- ❑ Promove interoperabilidade
- ❑ Três componentes centrais: **Resources, Tools e Prompts**
- ❑ A arquitetura cliente-servidor separa responsabilidades
- ❑ Em grafos de execução, o MCP atua como camada de abstração
- ❑ Segurança e controle são nativos ao protocolo